

AR Pacesetter — 技術スタック

iPhone + ARグラス（2デバイス構成）によるARランニング・ペーシングシステム

1. システム全体図



処理はすべてiPhone内で完結。グラスは表示専用のため専用SDKに依存せず、iOS標準の外部ディスプレイ出力で映像を送る。

2. レイヤ別の技術

レイヤ	技術	役割
iPhoneアプリ	SwiftUI / Swift · CoreLocation · HealthKit	GPS · センサー取得、画面遷移、ログ出力
AR描画エンジン	Unity 6 + URP / AR Foundation (ARKit)	3m前方アバターとHUDの描画・追従
ARグラス	USB-C 有線 · DisplayPort Alt Mode	無圧縮1080p映像の投影 (SDK非依存)

3. 技術的なキモ — 接合部

- **UaaL (Unity as a Library)**：1つのXcodeワークスペースに SwiftUIアプリと Unity を同居させ、Objective-C++ ブリッジで双方向通信。ネイティブUIの操作性とUnityの描画力を1アプリで両立。
- **グラス制御にベンダーSDKを使わない**：iOS標準の外部ディスプレイAPIで出力するため、グラスの機種依存を排除できる。

4. 品質を守る仕組み（実機なしで回る検証）

dotnet test	純ロジックのユニットテスト（Unity起動不要・数秒で完了）
Unityヘッドレス E2E	コマンド一発で全シナリオを回帰検証
GitHub Actions	macOSランナーで署名なしiOSビルド検証（実機を借りずに確認）

一言でいうと

SwiftUI（センサー・UI） + Unity（AR描画）を UaaL で1アプリに統合し、ARグラスへは標準の外部ディスプレイ出力で投影する。

動作環境：iPhone 12 Pro 以上 / iOS 26+ / 60fps · Motion-to-Photon 20ms 以内